

GRAIN-CAN: Same-ID Local Behavior Features for Cross-Vehicle CAN Intrusion Detection

Qi’ao Li

School of Cyber Science and Technology, Beihang University, Beijing, China
qi_aolee@buaa.edu.cn

Abstract. Cross-vehicle CAN intrusion detection is difficult because vehicle-specific message distributions and attack patterns can both shift between training and deployment. Raw fixed-window representations further obscure short attack bursts when only a few frames in a long window carry abnormal evidence. This paper proposes GRAIN-CAN, a supervised feature-based CAN IDS pipeline that exposes same-ID local behavior changes before window aggregation. For each CAN frame, GRAIN-CAN maintains a per-ID history state and extracts timing residuals, payload changes, payload statistics, and local ID-behavior features using only past traffic. These frame-level features are aggregated with a fixed pre-test window configuration and classified by a lightweight supervised detector. We evaluate GRAIN-CAN on CT&T test01–test04 using attack-positive precision, recall, F1, FNR, FPR, and confusion matrices, with AUPR and fixed-FPR detection rates reported as supplementary score-based metrics. The results show that same-ID local behavior features improve over raw fixed-window baselines in the hardest shifted settings, while ablations identify timing residuals as the strongest signal in the current setting.

Keywords: CAN bus · Intrusion detection · Automotive security · Feature pipeline

1 Introduction

Modern vehicles depend on CAN buses for communication among electronic control units, yet CAN was not designed with message authentication or origin verification. Practical vehicle-security studies have shown that this design leaves room for malicious message injection and remote attack paths [10, 4, 13]. CAN intrusion detection systems therefore monitor bus traffic and infer abnormal behavior from timestamps, arbitration IDs, DLCs, and payload bytes.

Cross-vehicle detection is harder than closed-split detection. A detector trained on one vehicle may see different ID frequencies, payload ranges, and timing profiles on another vehicle. Unknown-attack settings add a second shift: the attack family and script may also differ. Raw fixed-window detectors face an additional problem. If a short attack burst affects only a few frames inside a long window, the abnormal evidence can be diluted before classification.

GRAIN-CAN is built around a simple design insight: before constructing a window, convert each frame into local behavior changes relative to recent history of the same CAN ID. The detector keeps a per-ID history state, extracts timing residuals, payload-change residuals, payload statistics, and local ID-behavior features, then aggregates those features under a fixed pre-test window configuration. The supervised classifier is intentionally lightweight; the representation before aggregation is the contribution.

The paper makes two contributions. First, we propose GRAIN-CAN, a supervised feature-based CAN IDS pipeline that exposes same-ID local behavior before fixed-window aggregation. Second, we provide a controlled CT&T cross-shift evaluation that compares pipeline-level representations, including public-style classical features, raw fixed-window Transformer results, safe-feature baselines, and GRAIN windows, using conventional IDS metrics and feature ablations.

2 Background and Design Motivation

2.1 CAN Intrusion Detection Under Cross-Shift

CAN IDS methods use timing, payload evolution, ID frequencies, transition structure, learned sequence models, signal-level representations, or physical-layer fingerprints [15, 16, 5, 6]. Recent work has explored BERT-style sequence modeling, signal-level autoencoder frameworks, graph-style representations, federated learning, and Mamba-style sequence models [1, 14, 18, 2, 12]. Benchmark datasets such as ROAD and CT&T make cross-setting evaluation more concrete [17, 3, 11, 7].

2.2 Why Raw Fixed Windows Lose Local Attack Evidence

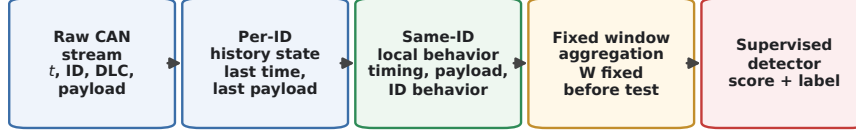
Many CAN attacks are local in both time and ID space. A spoofing or injection burst may alter only a handful of messages, while the surrounding window remains normal. A raw fixed-window sequence model must learn both the local perturbation and how to keep it visible after aggregation or pooling. This is possible, but the representation burden is high when the vehicle distribution changes.

2.3 Design Insight: Same-ID Local Behavior Changes

GRAIN-CAN starts from the observation that CAN IDs often carry repeated behavior. Comparing a frame with the recent history of the same ID exposes residuals: timing gaps, payload continuity changes, payload summary shifts, and local concentration changes. These residuals are not vehicle-independent, but they are less tied to absolute payload or frequency values than raw windows alone.

GRAIN-CAN detector pipeline

Same-ID local behavior is extracted before fixed-window classification.



History-only extraction; no future frames, no test labels, no test-fitted statistics.

Fig. 1. GRAIN-CAN is a detector pipeline: it converts raw CAN frames into same-ID local behavior features before fixed-window supervised classification.

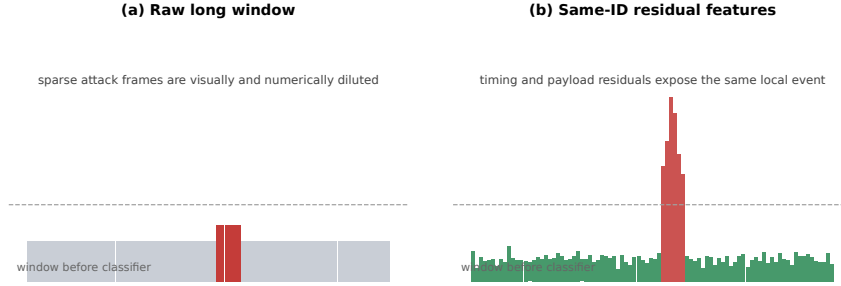


Fig. 2. Feature-before-window extraction keeps sparse local attack evidence visible before aggregation, whereas raw long windows can numerically dilute the same event.

3 GRAIN-CAN Pipeline

3.1 Overview

GRAIN-CAN has four stages: per-ID history maintenance, frame-level same-ID local behavior extraction, fixed pre-test window aggregation, and supervised classification. Unlike raw-window neural detectors that learn directly from a fixed sequence of frames, GRAIN-CAN first converts each frame into local behavior residuals relative to the recent history of the same CAN ID.

3.2 Input Stream and Per-ID History State

The input stream is $x_i = (t_i, id_i, dlc_i, b_i)$, where t_i is the timestamp, id_i is the arbitration ID, dlc_i is the data length, and b_i denotes the payload bytes. For every ID, GRAIN-CAN maintains a compact state consisting of the last timestamp, last payload, recent counters, and recent summary statistics. The state is updated online and uses only current and past frames.

3.3 Same-ID Local Behavior Feature Extraction

The extracted features are grouped by purpose. Timing residuals capture disruptions in periodicity or burst patterns. Payload-change residuals capture discontinuities in byte-level behavior. Payload statistics summarize coarse byte distribution without requiring DBC decoding. Local ID-behavior features capture short concentration and frequency changes. These groups are later aggregated into fixed windows.

3.4 Window Construction and Labeling

Frame features are aggregated into fixed windows such as W10, W20, or W100. The window size is a configuration fixed before testing. A window is labelled attack if any frame in the window is labelled attack. This rule is common for window-level IDS experiments, but it also means rare attack windows can contain many normal frames; the design of GRAIN-CAN is intended to keep the local evidence visible before the label is applied.

3.5 Supervised Detector and Score Output

In the repository experiments, GRAIN-CAN uses a gradient-boosting tree ensemble. The classifier is not the main contribution; it learns a joint decision boundary over interpretable same-ID local behavior features. When the classifier exposes a score, we report AUPR and fixed-FPR detection rates as supplementary operating evidence.

Table 1. Compared IDS pipelines. GRAIN-CAN differs in representation before aggregation rather than in classifier complexity.

Detector	Representation	Classifier	Purpose
Public GB	sample public-style features	GB	classical reproduced baseline
Public MLP	sample public-style features	MLP	neural classical baseline
Raw-Trans	raw fixed frame sequence, W100	Transformer	raw-window deep baseline
GRAIN-W100	same-ID local behavior features, W100	GB	proposed detector pipeline

3.6 Training and Inference Protocol

Algorithm 1: training. Given a fixed window size W , initialize an empty per-ID history table. Process the training stream in timestamp order. For each frame, read the current state of its CAN ID, compute local behavior features, update the state with the current frame, and append the feature vector to the current

window. After every W frames, aggregate the frame features into a window vector and assign an attack label if any frame in the window is attack. Fit the supervised detector on training windows and, if a score threshold is required, select it on validation data.

Algorithm 1: inference. Freeze the feature definitions, window size, detector, and threshold. Process the test stream once in timestamp order. For each frame, compute features from current and past traffic only, update the per-ID state, aggregate windows, and emit a score or label. Test labels are used only after inference for evaluation.

3.7 Relation to Existing Pipeline Types

GRAIN-CAN is not intended to replace signal-level, graph, self-supervised, or deep sequence IDS frameworks. Its scope is narrower: a lightweight supervised representation pipeline that exposes same-ID local behavior before aggregation and can be evaluated under cross-shift settings.

Table 2. Positioning against representative CAN IDS pipeline families.

Work	Core mechanism	Required information	Difference
CANShield	signal-level multi-scale autoencoder ensemble	DBC / signal extraction	GRAIN uses raw fields and no AE ensemble
X-CANIDS	signal-aware explainable detection	CAN database / signal semantics	GRAIN does not require decoded signals
CANet	message-level unsupervised neural detector	raw CAN messages	GRAIN is supervised and window-aggregated
MIDS	bidirectional Mamba sequence model	message sequence	GRAIN is a feature pipeline, not a deep backbone
GRAIN-CAN	same-ID local behavior before aggregation	timestamp, ID, DLC, payload	proposed lightweight supervised pipeline

4 Related Work

4.1 Rule, Statistical, and Timing CAN IDS

Early CAN IDS work often used message timing, ID frequency, and interval statistics because many injection attacks perturb periodic traffic. These approaches are attractive for deployment because they are lightweight and interpretable [15, 16, 3]. Their weakness is that a single statistic can miss attacks that preserve timing or shift behavior across vehicles. GRAIN-CAN keeps the

lightweight spirit but uses multiple local residual features jointly through a supervised classifier.

4.2 Deep Sequence and Representation Learning

Deep CAN IDS methods learn from raw message sequences, payload bits, language-model-style tokenization, or modern sequence backbones [1, 12]. These methods can learn rich dependencies but often require larger training sets, careful protocol design, and more compute. GRAIN-CAN takes a different route: it exposes local behavior changes with explicit features before any classifier sees a window.

4.3 Signal-Level and Explainable Frameworks

Signal-level frameworks such as CANShield and X-CANIDS use decoded signals or semantic signal structure to improve detection and interpretability [14, 9]. This is powerful when DBC or signal-level information is available. GRAIN-CAN uses only raw CAN fields and same-ID history, so it is easier to apply when signal definitions are unavailable.

4.4 Unsupervised and Historical Models

Unsupervised and historical models such as CANet learn normal message behavior without attack labels [8]. GRAIN-CAN is supervised and should not be framed as solving arbitrary zero-day detection. Its value is narrower: it tests whether history-based local residuals improve cross-shift supervised detection when labels exist.

5 Experimental Setup

5.1 Datasets and Cross-Shift Settings

CT&T set01 is the main dataset. Test01 uses a known vehicle and known attack families; Test02 changes the vehicle; Test03 changes the attack family; Test04 changes both. This gives a compact vehicle-shift and attack-shift matrix.

Table 3. CT&T cross-shift settings used in the main evaluation.

Setting	Vehicle shift	Attack shift	Frames	Positive rate	Role
Test01	known	known	16355810	0.007	known-vehicle known-attack baseline
Test02	unknown	known	17101057	0.012	vehicle-shift stress test
Test03	known	unknown	19288415	0.004	attack-shift stress test
Test04	unknown	unknown	23873695	0.003	joint vehicle/attack-shift stress test

5.2 Compared IDS Pipelines

The main comparison keeps only representative detectors: public-style classical GradientBoosting and MLP rows, a raw fixed-window Transformer, and GRAIN-CAN windows. Simple single-signal thresholds are treated as diagnostic checks outside the main figure.

5.3 Metrics and Threshold Rules

Attack is the positive class. We report precision, recall, F1, FNR, and FPR from the confusion matrix. AUPR, AUROC, and recall at fixed FPR are supplementary because they require comparable scores. Window size and threshold must be fixed before test evaluation.

5.4 Implementation Details

The GRAIN rows use history-only features over timestamp, ID, DLC, payload bytes, same-ID timing residuals, payload-change residuals, payload statistics, and local ID behavior. W10, W20, and W100 are reported as configurations rather than test-selected winners.

6 Evaluation

6.1 Cross-Shift Detection Results

Table 4 and Fig. 3 summarize the main cross-shift results. The key pattern is not that one detector wins every setting. Rather, GRAIN-CAN provides strong attack-positive detection under vehicle and attack shifts, especially compared with raw fixed-window behavior in Test02 and Test04.

Table 4. Main CT&T results with attack as the positive class.

Setting	Detector	P	R	F1	FNR	FPR
Test01	GB-sample	0.860	0.980	0.916	0.020	0.001
Test01	MLP	0.209	0.365	0.266	0.635	0.010
Test01	Raw-Trans	1.000	0.931	0.964	0.069	0.000
Test01	GRAIN-W100	0.999	0.892	0.942	0.108	0.000
Test02	GB-sample	0.931	0.998	0.964	0.002	0.001
Test02	MLP	0.283	0.933	0.434	0.067	0.030
Test02	Raw-Trans	0.087	1.000	0.160	0.000	0.133
Test02	GRAIN-W100	0.984	0.931	0.956	0.069	0.000
Test03	GB-sample	0.719	0.629	0.671	0.371	0.001
Test03	MLP	0.087	0.574	0.151	0.426	0.022
Test03	Raw-Trans	1.000	0.013	0.025	0.987	0.000
Test03	GRAIN-W100	1.000	0.662	0.797	0.338	0.000
Test04	GB-sample	0.707	0.447	0.548	0.553	0.000
Test04	MLP	0.002	0.077	0.004	0.923	0.089
Test04	Raw-Trans	0.011	0.980	0.022	0.020	0.229
Test04	GRAIN-W100	0.938	0.519	0.668	0.481	0.000

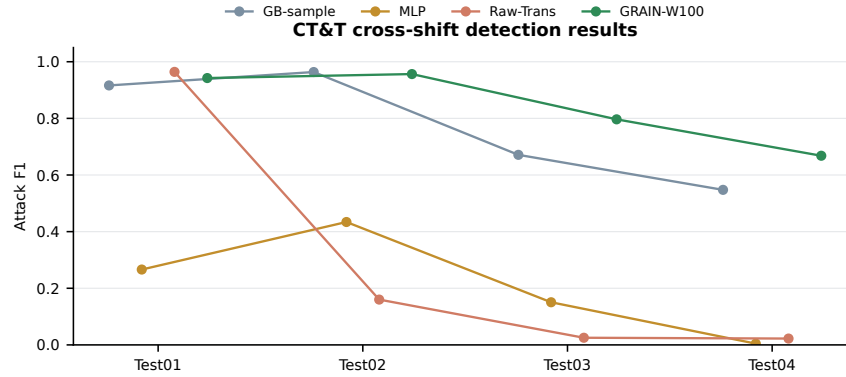


Fig. 3. GRAIN-CAN remains competitive across CT&T shifts, while the raw fixed-window Transformer collapses in vehicle-shifted settings.

6.2 Effect of Feature-Before-Window Representation

The raw-window comparison isolates the representation effect. Both pipelines use fixed windows, but only GRAIN-CAN extracts same-ID local behavior before aggregation. The contrast is strongest in Test02 and Test04, where raw windows produce high miss or false-alarm behavior.

Table 5. Raw fixed-window pipeline versus GRAIN-CAN.

Setting	Pipeline	P	R	F1	AUPR
Test01	GRAIN-W100	0.999	0.892	0.942	0.896
Test01	Raw-Trans	1.000	0.931	0.964	0.984
Test02	GRAIN-W100	0.984	0.931	0.956	0.935
Test02	Raw-Trans	0.087	1.000	0.160	0.049
Test03	GRAIN-W100	1.000	0.662	0.797	0.925
Test03	Raw-Trans	1.000	0.013	0.026	0.271
Test04	GRAIN-W100	0.936	0.519	0.668	0.785
Test04	Raw-Trans	0.011	0.979	0.022	0.173

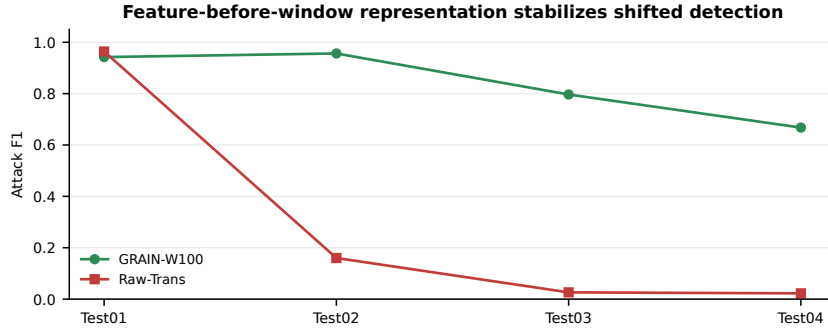


Fig. 4. The same W100 aggregation becomes more reliable when local behavior residuals are computed before the window is formed.

6.3 Contribution of Local Behavior Features

Feature-group ablation on Test04 shows that same-ID timing is the strongest individual signal in the current setting. Removing same-ID timing sharply degrades F1, while payload and ID groups have weaker effects. This supports the pipeline’s focus on residual behavior rather than absolute payload values alone.

Table 6. Feature-group ablation on CT&T Test04.

Variant	P	R	F1	FNR	FPR	AUPR	R@1e-3
Timing only	0.771	0.491	0.600	0.509	0.000	0.425	0.491
w/o CAN ID	0.740	0.355	0.480	0.645	0.000	0.265	0.355
Full safe features	0.740	0.354	0.479	0.646	0.000	0.255	0.355
w/o payload stats	0.740	0.354	0.479	0.646	0.000	0.255	0.355
w/o payload delta	0.740	0.354	0.479	0.646	0.000	0.255	0.355
w/o same-ID timing	0.005	0.249	0.009	0.751	0.056	0.057	0.067

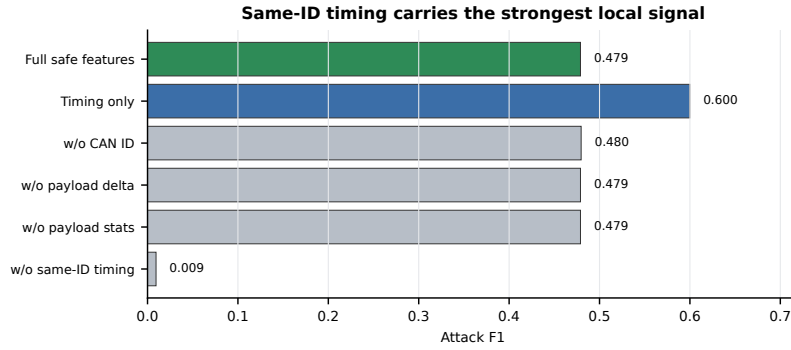


Fig. 5. Ablation identifies same-ID timing residuals as the strongest local behavior signal in Test04.

6.4 Sensitivity to Aggregation Window

W10, W20, and W100 behave differently across settings. Shorter windows help Test01 and Test02, while W100 is stronger in Test03 and Test04. The result argues for reporting window sensitivity instead of treating one test-selected window as a universal hyperparameter.

6.5 Low-False-Alarm Operating Points

Score-based operating analysis is supplementary. Fig. 6 compares AUPR and the available fixed-FPR detection evidence on Test04. These metrics are useful for deployment discussions, but they are reported only when the underlying score is available under the same protocol.

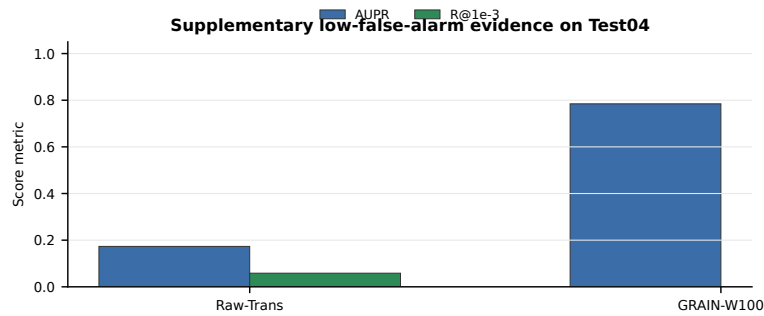


Fig. 6. Supplementary Test04 operating evidence shows that AUPR and fixed-FPR recall must be interpreted separately from thresholded F1.

6.6 Diagnostic Rule and Statistical Checks

We include simple single-signal detectors only as diagnostic checks. They test whether GRAIN-CAN is merely a single threshold over timing or payload change. They are not presented as recent state-of-the-art competitors.

7 Discussion

7.1 What GRAIN-CAN Explains

The experiments support the design claim that local same-ID behavior should be made explicit before window aggregation. GRAIN-CAN is strongest when raw windows struggle to preserve sparse evidence under vehicle or attack shifts.

7.2 Failure Modes

GRAIN-CAN still needs supervised labels and visible disruptions in timing, payload continuity, or local ID behavior. Attacks that preserve those properties can be missed. The method is also not vehicle-independent: it reduces reliance on absolute distributions but does not remove distribution shift.

7.3 Comparability to Prior Work

Prior reported aggregate metrics cannot be directly compared with attack-positive F1 unless the positive class, averaging rule, thresholds, and confusion matrix are available. We therefore compare reproduced local rows under a common metric definition and treat score metrics as supplementary.

8 Conclusion

GRAIN-CAN demonstrates that same-ID local behavior extraction before window aggregation is a useful lightweight design for cross-shift CAN IDS. Its strength is interpretable representation-level robustness; its limitation is that supervised labels and visible timing or payload disruptions are still required. Future work should combine this representation with self-supervised training, signal-level features, DBC-aware variants, online adaptation, and real-time deployment evaluation.

References

1. Alkhatib, N., Mushtaq, M., Ghauch, H., Danger, J.L.: CAN-BERT do it? controller area network intrusion detection system based on BERT language model. arXiv preprint arXiv:2210.09439 (2022)
2. Althunayyan, M., Javed, A., Rana, O.: A robust multi-stage intrusion detection system for in-vehicle network security using hierarchical federated learning. arXiv preprint arXiv:2408.08433 (2024)

3. Blevins, D.H., Moriano, P., Bridges, R.A., Verma, M.E., Iannacone, M.D., Hollifield, S.C.: Time-based CAN intrusion detection benchmark. *arXiv preprint arXiv:2101.05781* (2021)
4. Checkoway, S., McCoy, D., Kantor, B., Anderson, D., Shacham, H., Savage, S., Koscher, K., Czeskis, A., Roesner, F., Kohno, T.: Comprehensive experimental analyses of automotive attack surfaces. In: *Proceedings of the USENIX Security Symposium* (2011)
5. Cho, K.T., Shin, K.G.: Fingerprinting electronic control units for vehicle intrusion detection. In: *Proceedings of the USENIX Security Symposium*. pp. 911–927 (2016)
6. Cho, K.T., Shin, K.G.: Viden: Attacker identification on in-vehicle networks. In: *Proceedings of the ACM Conference on Computer and Communications Security*. pp. 1109–1123 (2017)
7. Guerra, L., Xu, L., Bellavista, P., Chapuis, T., Duc, G., Mozharovskiy, P., Nguyen, V.T.: AI-driven intrusion detection systems on the ROAD dataset: A comparative analysis for automotive CAN. *arXiv preprint arXiv:2408.17235* (2024)
8. Hanselmann, M., Strauss, T., Dormann, K., Ulmer, H.: CANet: An unsupervised intrusion detection system for high dimensional CAN bus data. *arXiv preprint arXiv:1906.02492* (2019)
9. Jeong, S., Lee, S., Lee, H., Kim, H.K.: X-CANIDS: Signal-aware explainable intrusion detection system for controller area network-based in-vehicle network. *arXiv preprint arXiv:2303.12278* (2023)
10. Koscher, K., Czeskis, A., Roesner, F., Patel, S., Kohno, T., Checkoway, S., McCoy, D., Kantor, B., Anderson, D., Shacham, H., Savage, S.: Experimental security analysis of a modern automobile. In: *Proceedings of the IEEE Symposium on Security and Privacy*. pp. 447–462 (2010)
11. Lampe, B., Meng, W.: can-train-and-test: A curated CAN dataset for automotive intrusion detection. *Computers & Security* **140**, 103777 (2024)
12. Liu, Q., Song, R., Cui, L., Zhang, H., Sun, Y., Sun, L.: MIDS: Detecting stealthy masquerade and tampering attacks on CAN bus via bidirectional Mamba. *arXiv preprint arXiv:2606.18599* (2026)
13. Miller, C., Valasek, C.: Remote exploitation of an unaltered passenger vehicle. In: *Black Hat USA* (2015)
14. Shahriar, M.H., Xiao, Y., Moriano, P., Lou, W., Hou, Y.T.: CANShield: Deep learning-based intrusion detection framework for controller area networks at the signal-level. *arXiv preprint arXiv:2205.01306* (2022)
15. Song, H.M., Kim, H.R., Kim, H.K.: Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network. In: *International Conference on Information Networking*. pp. 63–68 (2016)
16. Taylor, A., Leblanc, S., Japkowicz, N.: Frequency-based anomaly detection for the automotive CAN bus. In: *World Congress on Industrial Control Systems Security*. pp. 45–49 (2015)
17. Verma, M.E., Bridges, R.A., Hollifield, S.C., Iannacone, M.D., Moriano, P.: ROAD: The real ORNL automotive dynamometer controller area network intrusion detection dataset. *arXiv preprint arXiv:2012.14600* (2020)
18. Wang, K., Jiang, Q., Wang, B., Zhang, Y., Wu, Y.: Effective in-vehicle intrusion detection via multi-view statistical graph learning on CAN messages. *arXiv preprint arXiv:2311.07056* (2023)